

Ship Classification Challenge 2026: A Hand-Written CNN for 10-Class Greyscale Recognition

Aymeric Hiegel

August 2026

1 Abstract

Automatic recognition of vessel types from low-resolution greyscale imagery is a representative small-data computer vision problem: the classes are visually close, the only usable cue is the silhouette, and colour information is absent by construction. In this project we address a 10-class classification problem over 37,980 greyscale images of size 128×192 , under the explicit constraint that the network must be written by hand and must contain fewer than 30 layers. We design DeepGAPCNN, a fully convolutional architecture built from repeated 3×3 convolution blocks with batch normalisation, terminated by global average pooling, and we train it from scratch with class-weighted cross-entropy. Starting from a baseline of 92.21% accuracy on a held-out internal test split, we study four regularisation strategies in isolation (label smoothing, strong augmentation, MixUp [9], CutMix [10]), quantify their error diversity, and combine them by ensembling and test-time augmentation, reaching 93.42%. We then show that the limiting factor was capacity rather than regularisation: densifying the network to DeepGAPCNN_v2 (12.2M parameters, 13 layers) makes strong augmentation and MixUp *profitable* instead of harmful, lifting a single model to 96.92%, and a five-seed deep ensemble [12] to 97.39%. All figures reported in this document are measured on an internal stratified test split of 3,798 images; no leaderboard score is available from the experimental record.

2 Introduction

2.1 Context and constraints

The task is posed as a Kaggle-style competition: predict the class of every image in a hidden test set and submit a `test.csv` file. The grading is based on the ranking obtained on that hidden set. Two constraints shape the whole project:

- **Hand-written network.** The convolutional network must be implemented explicitly, layer by layer. Pre-trained backbones and off-the-shelf architectures are excluded, which rules out transfer learning entirely: every model discussed here is trained from random initialisation.
- **Fewer than 30 layers.** The number of layers is checked explicitly. This caps the depth budget and makes the choice of *where* to spend capacity a design decision rather than an afterthought.

These constraints are more restrictive than they first appear. Without pre-training, the network cannot import any prior about edges, textures or shapes; it must learn the notion of a hull silhouette from 30,384 training images alone. This is the main reason why regularisation behaves counter-intuitively in Section 5.

2.2 Objective

The objective is to maximise classification accuracy on the hidden test set while respecting the two constraints above. Because the hidden set gives no feedback during development, all methodological decisions in this report are

Table 1: Class distribution in ships_gray/.

Idx	Class	Images	%
1	containership	6,266	16.50
3	cruiser	6,251	16.46
5	destroyer	5,981	15.75
6	methanier	3,172	8.35
0	coastguard	3,128	8.24
9	tug	3,104	8.17
7	smallfish	3,014	7.94
2	corvette	2,676	7.05
8	submarine	2,562	6.75
4	cv	1,826	4.81
Total		37,980	100.0

driven by a stratified internal split. The working method is deliberately incremental: one modification at a time, each measured against the previous configuration, so that every gain (or loss) can be attributed to a single cause.

2.3 Dataset

The training material is the `ships_gray/` directory: 37,980 single-channel images, all of identical size 192×128 pixels (width $\times$ height), organised in ten class folders. Image dimensions were verified empirically on a 200-image sample rather than assumed. The evaluation material is `test_images.npy`, an array of shape $(4224, 128, 192)$ with `uint8` values, for which no labels are provided.

The ten classes and their frequencies are given in Table 1 and illustrated in Figure 1. The distribution is imbalanced but not pathologically so: the ratio between the most frequent class (`containership`, 6,266 images) and the rarest (`cv`, 1,826 images) is 3.4:1.

Figure 2 shows one sample per class and explains where the difficulty lies. The images are rendered consistently, with the vessel seen broadside and centred, so background variation is negligible. What separates the classes is almost entirely the outline. Several classes are naval surface combatants of comparable size (`corvette`, `destroyer`, `cruiser`), and their silhouettes differ only by superstructure detail that survives poorly at 128×192 . This intuition is confirmed quantitatively by the confusion analysis in

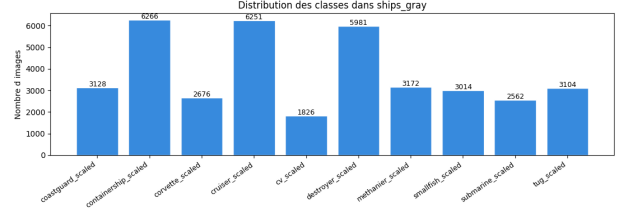


Figure 1: Class distribution in ships_gray/. The imbalance is moderate: three majority classes (`containership`, `cruiser`, `destroyer`) account for roughly half of the dataset, while `cv` contributes under 5%.

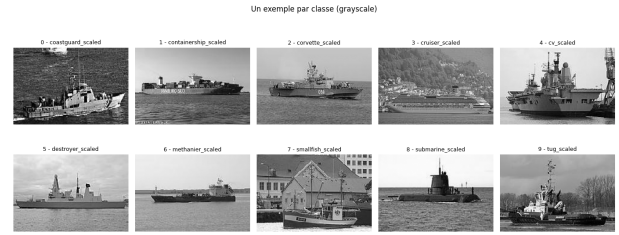


Figure 2: One random sample per class. All images share the same greyscale rendering and a broadly similar view-point, so the discriminative information is concentrated in the silhouette: hull length, superstructure height, and deck profile.

Section 4.3.

3 Methodological approach

3.1 Data partitioning and normalisation

3.1.1 Stratified split

The 37,980 labelled images are split into three disjoint subsets by two successive stratified draws with a fixed seed (42): first 10% is held out as an internal test set, then 1/9 of the remainder becomes the validation set. The resulting sizes are 30,384 / 3,798 / 3,798 for train, validation and test respectively, i.e. an 80/10/10 partition.

Stratification is applied on the labels at both stages, so the class proportions are preserved to within 0.02 percentage points across the three subsets (Table 2). The roles

Table 2: Class proportions after the stratified split.

Class	Train	Val	Test
coastguard	8.23%	8.24%	8.24%
containership	16.50%	16.51%	16.51%
corvette	7.04%	7.06%	7.06%
cruiser	16.46%	16.46%	16.46%
cv	4.81%	4.82%	4.82%
destroyer	15.75%	15.75%	15.75%
methanier	8.35%	8.35%	8.35%
smallfish	7.94%	7.93%	7.93%
submarine	6.75%	6.74%	6.74%
tug	8.18%	8.16%	8.16%
Size	30,384	3,798	3,798

are strictly separated: the validation set drives model selection and early stopping, the internal test set is used only for the final reported figures.

3.1.2 Normalisation statistics

Rather than reusing generic ImageNet statistics — which would be meaningless for single-channel synthetic-looking imagery — the normalisation constants are estimated on the training split only. A first pass accumulates the pixel sum to obtain the mean, a second pass accumulates the squared deviation to obtain the standard deviation:

$$\mu = \frac{1}{N} \sum_i x_i, \quad \sigma = \sqrt{\frac{1}{N} \sum_i (x_i - \mu)^2}, \quad (1)$$

where N is the total number of pixels in the training split. This yields $\mu = 0.5420$ and $\sigma = 0.2420$. Computing the statistics on the training split alone avoids leaking information from validation and test into the preprocessing pipeline. After normalisation a training batch spans $[-2.24, 1.89]$.

3.2 Data augmentation

3.2.1 Baseline augmentation

The default training pipeline applies, in order: random horizontal flip ($p = 0.5$), a random affine transform (rotation $\pm 8^\circ$, translation 5%, scale $[0.92, 1.08]$), colour jitter

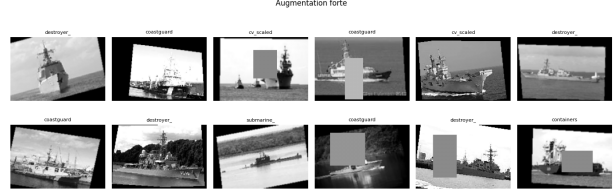


Figure 3: Twelve samples drawn from the strong augmentation pipeline (de-normalised for display). Blur, perspective and large erased patches make individual samples substantially harder than the originals.

on brightness and contrast (0.2 each), tensor conversion, normalisation, and finally random erasing ($p = 0.25$, area $[0.02, 0.10]$).

Two choices deserve comment. Horizontal flip is safe here because a vessel photographed bow-left and the same vessel bow-right are both legitimate members of the class; vertical flip would not be, since it would put the superstructure under the waterline. The rotation range is kept deliberately narrow ($\pm 8^\circ$) because the waterline is a strong horizontal reference in these images, and destroying it removes a cue the network can otherwise rely on.

3.2.2 Strong augmentation

A more aggressive variant widens every parameter and adds two operators: rotation $\pm 15^\circ$, translation 10%, scale $[0.82, 1.18]$, shear 5° , random perspective (distortion 0.15, $p = 0.3$), colour jitter 0.3, Gaussian blur ($\sigma \in [0.1, 1.5]$), and random erasing with $p = 0.5$ over an area of up to 20%. Figure 3 shows the result: silhouettes are noticeably degraded, and the erased patches occasionally remove a discriminative part of the superstructure.

3.3 Handling class imbalance

The 3.4:1 imbalance is compensated inside the loss rather than by resampling, using inverse-frequency weights normalised so that they average to one:

$$w_c = \frac{N_{\text{train}}}{C \cdot n_c}, \quad (2)$$

with $C = 10$ classes and n_c the number of training images of class c . The resulting weights range from 0.606

Table 3: DeepGAPCNN architecture. Every ConvBlock is CONV3×3–BN–ReLU–CONV3×3–BN–ReLU, optionally followed by 2×2 max pooling.

Stage	Channels	Output
Input	1	128×192
Block 1 + pool	32	64×96
Block 2 + pool	64	32×48
Block 3 + pool	128	16×24
Block 4 + pool	256	8×12
Block 5	512	8×12
GAP	512	1×1
Dropout 0.3	512	—
Linear	10	10

for containership to 2.081 for cv. These weights are passed to CrossEntropyLoss for training. Validation loss is computed with the *unweighted* criterion, so that model selection optimises the metric actually being graded rather than the reweighted surrogate.

3.4 Architecture: DeepGAPCNN

The network, summarised in Table 3, is a VGG-style stack of five blocks. Each block applies two 3×3 convolutions, each followed by batch normalisation [2] and ReLU, and the first four blocks end with 2×2 max pooling. Convolutions carry no bias term, which is redundant in front of batch normalisation.

Three design decisions are worth justifying.

Global average pooling. The classifier head is a global average pool followed by a single linear layer, in the spirit of Network in Network [1]. Flattening the final $8 \times 12 \times 512$ map into a dense layer would cost roughly 49M parameters for the head alone; GAP reduces this to 5,130 and removes the dominant source of overfitting. It also makes the network insensitive to the exact spatial position of the hull, which is desirable since translation is part of the augmentation.

Channel schedule. Channels double at every stage ($32 \rightarrow 512$) while spatial resolution is halved, keeping the per-stage computational cost roughly balanced. The

Table 4: Training hyperparameters for the baseline model.

Parameter	Value
Optimiser	Adam [5]
Learning rate	2×10^{-3}
Weight decay	10^{-4}
Scheduler	CosineAnnealingLR [7]
Batch size	1024
Epochs	30
Loss	weighted cross-entropy
Mixed precision	bf16 autocast
Early stopping	patience 20 (val. accuracy)
Hardware	NVIDIA A100-PCIE-40GB

fifth block deliberately omits pooling: at 8×12 the map is already small, and pooling further would discard super-structure detail before the GAP averages it away.

Layer budget. Counting Conv2d and Linear modules, the network uses 11 layers, well inside the 30-layer limit imposed by the assignment. It totals 4,718,762 trainable parameters.

3.5 Training protocol

Table 4 lists the baseline settings. Training uses bf16 automatic mixed precision, which on an A100 roughly halves memory traffic; the wide dynamic range of bf16 makes gradient scaling unnecessary, unlike float16. The large batch size of 1024 is chosen to keep the GPU saturated, and is paired with a correspondingly large learning rate.

Model selection tracks validation accuracy: the state dictionary is snapshotted whenever validation accuracy improves, and the best snapshot — not the final epoch — is restored at the end of training. This matters, because the validation curve is markedly non-monotonic in the early epochs (Section 4.1).

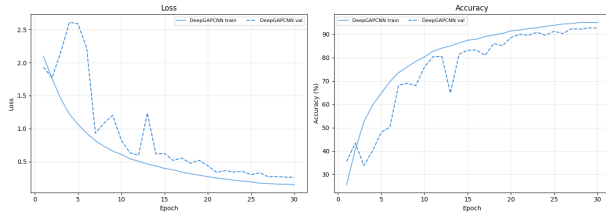


Figure 4: Loss (left) and accuracy (right) for DeepGAPCNN over 30 epochs. Solid: training; dashed: validation. The validation loss is strongly unstable before epoch 15 and converges only as the cosine schedule anneals the learning rate.

4 Baseline results

4.1 Learning curves

Figure 4 shows the 30-epoch run. Three phases are visible. Until epoch 7 the validation loss is not merely noisy but *diverges*, peaking at 2.61 at epoch 4 while the training loss decreases monotonically. This is the expected signature of a large batch combined with an aggressive learning rate: the batch-normalisation running statistics lag behind the rapidly moving weights, so the model in `eval` mode is temporarily mismatched with the model in `train` mode. From epoch 7 to epoch 20 the two curves converge, with residual spikes at epochs 9 and 13. After epoch 20 the cosine schedule has reduced the learning rate enough that both curves become smooth.

The best validation accuracy, 92.71%, is reached at epoch 29; early stopping never triggers within the 30-epoch budget. The gap between final training and validation accuracy is only 2.37 points, indicating that the model is capacity-limited rather than overfitting — an observation that turns out to be the key to Section 7.

4.2 Overall performance

On the internal test split the baseline reaches 92.21% accuracy in 459 s of training.¹ Macro-averaged precision and

¹The notebook’s later summary tables carry a hard-coded baseline value of 93.13% inherited from an earlier run. The figure reported here, 92.21%, is the one actually measured by `evaluate()` and is consistent with the comparison table, the diversity analysis and the classification report, which all give 92.21% / 0.922.

Table 5: Per-class classification report for DeepGAPCNN on the internal test split.

Class	Prec.	Rec.	F1	Supp.
coastguard	0.831	0.850	0.840	313
containership	0.985	0.968	0.977	627
corvette	0.738	0.832	0.782	268
cruiser	0.989	0.971	0.980	625
cv	0.926	0.891	0.908	183
destroyer	0.943	0.880	0.910	598
methanier	0.962	0.959	0.961	317
smallfish	0.899	0.950	0.924	301
submarine	0.863	0.934	0.897	256
tug	0.934	0.906	0.920	310
Accuracy			0.922	3,798
Macro avg	0.907	0.914	0.910	3,798
Weighted avg	0.925	0.922	0.923	3,798

recall are 0.907 and 0.914; the macro F1 of 0.910 sits 1.3 points below the weighted F1 of 0.923, which quantifies the residual penalty paid on the minority classes despite the loss weighting.

4.3 Confusion matrix analysis

Figure 5 and Table 5 localise the errors precisely, and the picture is highly structured rather than diffuse.

Well-recognised classes. Merchant and civilian vessels are almost solved: `cruiser` (F1 0.980), `containership` (0.977) and `methanier` (0.961). These have long, flat, distinctive hulls with no close neighbour in the label set, and they are also the best-represented classes.

The surface-combatant cluster. The weakest class by a clear margin is `corvette` (F1 0.782, precision 0.738), and its errors are not random. Of the 268 test corvettes, 15 are predicted `coastguard` and 17 `destroyer`; symmetrically, 49 of the 598 destroyers — 8.2% of the class — are predicted `corvette`, which is the single largest off-diagonal cell in the whole matrix. The same group contains the `coastguard/corvette` pair (15 errors in the other direction). These are all grey-hulled military vessels differing mainly in length and superstructure count, and at

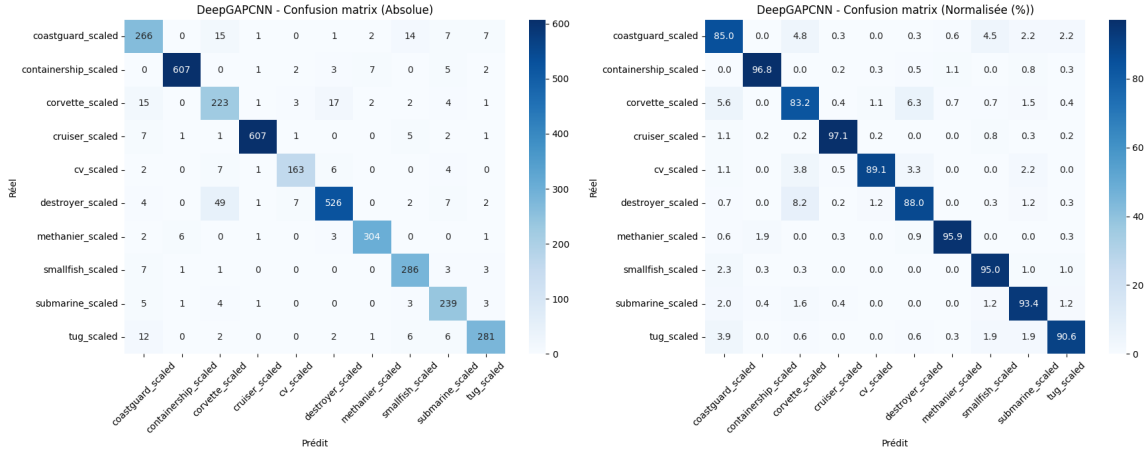


Figure 5: Confusion matrices for DeepGAPCNN on the internal test split (3,798 images). Left: absolute counts. Right: row-normalised percentages, i.e. per-class recall on the diagonal. The dominant off-diagonal cells all lie inside the surface-combatant group.

128×192 the distinguishing detail spans only a handful of pixels. The precision drop on *corvette* is a direct consequence of the class weighting: with $w = 1.42$ the loss actively pushes the model to over-predict this class, converting destroyer recall (0.880, the second-lowest) into corvette false positives.

Minority classes. *cv* (aircraft carriers), the rarest class at 4.81%, nonetheless achieves F1 0.908: its flight deck is a morphologically unambiguous cue, and rarity alone is not a handicap when the class is visually separable. *submarine* reaches recall 0.934 with precision 0.863, again a weighting-induced asymmetry.

The overall conclusion is that the residual error is concentrated in one biologically coherent group — naval surface combatants — and reflects a genuine limit of the input resolution rather than a failure of optimisation.

5 Single-model optimisations

Having established the baseline, we evaluate four regularisation strategies. Each is applied cumulatively on top of the previous one, following the sequence used during development, and each is trained from scratch with the

Adam recipe (learning rate lowered to 10^{-3}) so that the comparison is not confounded by initialisation.

5.1 Label smoothing

Motivation. Given the systematic corvette/destroyer confusion, a hard one-hot target forces the network to become fully confident on pairs that are intrinsically ambiguous. Label smoothing with $\varepsilon = 0.1$ replaces the target with $1 - \varepsilon$ for the true class and $\varepsilon/(C - 1)$ elsewhere, discouraging over-confident logits.

Result. Best validation accuracy rises from 92.71% to 93.36%, the largest validation gain of any single modification. Test accuracy, however, is 92.92% — an improvement over the baseline’s 92.21%, but a smaller one than the validation curve suggested. The training curve is also visibly smoother: the validation loss no longer diverges as violently in the first epochs, because the smoothed targets bound the loss magnitude.

5.2 Strong augmentation

Motivation. The baseline shows a train/validation gap of only 2.37 points, which is weak evidence for overfitting; nevertheless, the standard next step is to increase

augmentation strength, and the pipeline of Section 3.2.2 was applied on top of label smoothing over 35 epochs.

Result. Test accuracy *drops* to 90.60%, and best validation accuracy falls to 91.28%. Training accuracy stalls at 90.29% after 35 epochs, well below the 95.80% reached with baseline augmentation.

Diagnosis. This is underfitting, not a bad augmentation policy. With 4.7M parameters and no pre-training, the network does not have the capacity to absorb the extra invariance being demanded of it: Gaussian blur and 20% random erasing destroy exactly the fine superstructure detail that Section 4.3 identified as the only signal separating the surface combatants. The model is being asked to become invariant to the feature it needs. This diagnosis is what later motivates Section 7.

5.3 MixUp

Motivation. MixUp [9] constructs virtual training examples by convex interpolation of both inputs and targets:

$$\begin{aligned}\tilde{x} &= \lambda x_i + (1 - \lambda)x_j, \\ \mathcal{L} &= \lambda \mathcal{L}(f(\tilde{x}), y_i) + (1 - \lambda) \mathcal{L}(f(\tilde{x}), y_j),\end{aligned}\quad (3)$$

with $\lambda \sim \text{Beta}(\alpha, \alpha)$ and $\alpha = 0.2$. It encourages linear behaviour between training examples and is a natural complement to label smoothing.

Result. 90.57% test accuracy after 40 epochs (best validation 91.42%), statistically indistinguishable from strong augmentation alone and still 1.6 points below the baseline. Training accuracy under MixUp is reported against the mixed targets and reaches only 77.54%, confirming that the effective task has become much harder.

5.4 CutMix

Motivation. CutMix [10] replaces a rectangular region of one image with the corresponding region of another, with λ set to the complement of the pasted area fraction:

$$\lambda = 1 - \frac{(x_2 - x_1)(y_2 - y_1)}{H \times W}. \quad (4)$$

Unlike MixUp it preserves local spatial statistics, which should suit silhouette recognition better.

Table 6: Single-model regularisation study. All figures are accuracies on the internal test split.

Configuration	Best val.	Test
DeepGAPCNN (baseline)	92.71%	92.21%
+ label smoothing	93.36%	92.92%
+ strong augmentation	91.28%	90.60%
+ MixUp ($\alpha = 0.2$)	91.42%	90.57%
+ CutMix ($\alpha = 1.0$)	88.99%	88.02%

Result. 88.02%, the worst configuration tested, 4.2 points below the baseline. With $\alpha = 1.0$ the pasted rectangle is large and frequently covers the bow or the superstructure entirely, leaving an image whose visible content genuinely belongs to the other class while the loss still asks for a mixture. At this capacity the network cannot reconcile the two.

5.5 Cumulative results

Table 6 summarises the study. The result is unambiguous and worth stating plainly: **only label smoothing helps**, and every augmentation-based regulariser costs between 1.6 and 4.2 accuracy points. The standard recipe — pile on regularisation until validation stops improving — fails here, because the underlying problem is not variance but bias. The four degraded models are nevertheless not wasted: they are trained under genuinely different objectives, which makes them candidates for ensembling.

6 Diversity, TTA and ensembling

6.1 Diversity analysis

Ensembling only pays off when the members make *different* mistakes. We therefore quantify diversity before combining anything, using two complementary measures shown in Figure 6: the pairwise disagreement rate, and the Pearson correlation between binary error masks.

Disagreement ranges from 6.4% (baseline vs. label smoothing — the two most similar recipes) to 10.8% (label smoothing vs. CutMix). Error correlation tells the same story more sharply: the most complementary pair is +LS / +CutMix at $\rho = 0.496$, followed by

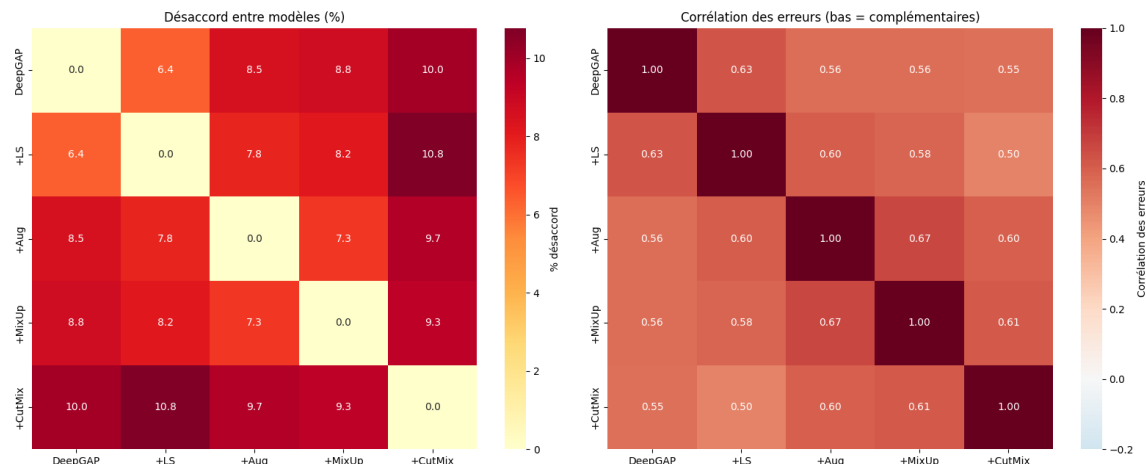


Figure 6: Left: pairwise disagreement rate between the five single models (% of test images on which two models predict different classes). Right: Pearson correlation between their binary error masks; lower values indicate more complementary models. Axis labels are those produced by the notebook.

DeepGAP / +CutMix (0.552), DeepGAP / +Aug (0.556) and DeepGAP / +MixUp (0.558); the least complementary pair is baseline vs. label smoothing at $\rho = 0.63$.

The practically important observation is that CutMix, the *worst* individual model at 88.02%, is also the most complementary one. Its errors are the least correlated with everyone else’s, which is precisely the property that makes an ensemble member useful. Individual accuracy is therefore not a sufficient criterion for ensemble membership.

6.2 Test-time augmentation

Test-time augmentation averages the softmax outputs over four views of each image: identity, horizontal flip, and ± 6 -pixel horizontal shifts with replicate padding. All four transformations are label-preserving for this dataset, and the shift is horizontal only, so the waterline reference is never disturbed.

Table 7 shows a clean monotone pattern: TTA *hurts* the two strongest models (-0.71 and -0.13 points) and *helps* the three weakest ones ($+0.18$, $+0.45$, $+0.50$). The interpretation follows directly from the training recipes. Models trained with heavy augmentation have already learned the corresponding invariances, so averaging over

Table 7: Effect of 4-view test-time augmentation on each single model.

Model	No TTA	TTA	Δ
DeepGAPCNN [†]	93.13%	92.42%	-0.71
+ LS	92.92%	92.79%	-0.13
+ LS + Aug	90.60%	90.78%	$+0.18$
+ MixUp	90.57%	91.02%	$+0.45$
+ CutMix	88.02%	88.52%	$+0.50$

[†] Reference value as recorded in the notebook; see the footnote in Section 4.2.

augmented views reduces prediction variance at no cost in bias. The baseline, trained with mild augmentation, is not shift-invariant, so the ± 6 -pixel views are effectively out-of-distribution inputs whose predictions dilute a correct one. TTA is a variance-reduction technique, and it only pays when there is variance to reduce.

6.3 Reasoned ensembling

Four combinations were evaluated by uniform averaging of softmax probabilities (Table 8). The best is E4, which selects the four best individual models — baseline, +LS,

Table 8: Ensembles of the five single models, by uniform averaging of softmax probabilities.

Ensemble	N	No TTA	TTA
E1 strategies only	4	92.58%	92.42%
E2 strategies + baseline	5	93.21%	93.26%
E3 all models	5	93.21%	93.26%
E4 top-4	4	93.42%	93.42%

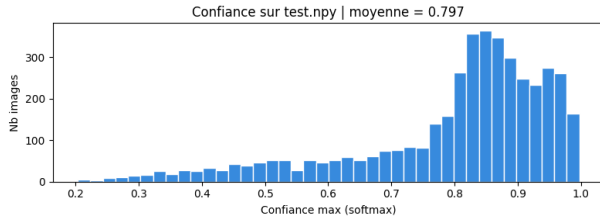


Figure 7: Distribution of the maximum softmax probability of ensemble E4 over the 4,224 competition images. Mean confidence 0.797; 345 predictions (8.2%) fall below 0.5.

+Aug, +MixUp — and reaches 93.42% both with and without TTA.

Two contrasts are informative. E1, which excludes the baseline, scores 92.58%, i.e. 0.6 points below E2 which is identical plus the baseline: the strongest member carries the ensemble. Conversely E4 beats E2 by 0.21 points by *dropping* CutMix, despite CutMix being the most diverse member according to Section 6.1. Diversity is necessary but not sufficient: at 88.02%, CutMix is simply too weak, and its confident wrong predictions outweigh the decorrelation benefit in a uniform average. A weighted average might have recovered it, but this was not explored.

The overall gain from ensembling, 93.42% versus 92.92% for the best single model, is +0.50 points — real, but modest for a fourfold increase in inference cost.

Ensemble E4 with TTA was used to produce the first test.csv submission over the 4,224 competition images. Figure 7 shows the confidence profile: the distribution is strongly bimodal, with a dominant mode above 0.8 and a long low-confidence tail. Mean maximum probability is 0.797 and 345 images (8.2%) are predicted with confidence below 0.5. That 8.2% is close to the 6.6% error rate

Table 9: DeepGAPCNN_v2. Same block structure as v1, one extra stage and a widened channel schedule.

Stage	Channels	Output
Input	1	128×192
Block 1 + pool	64	64×96
Block 2 + pool	128	32×48
Block 3 + pool	256	16×24
Block 4 + pool	384	8×12
Block 5 + pool	512	4×6
Block 6	512	4×6
GAP	512	1×1
Dropout	512	—
Linear	10	10

measured on the internal split, which is a reassuring consistency check: the uncertain fraction on the unlabelled competition data is of the same order as the observed error rate, suggesting no gross distribution shift between the two sets.

7 Densifying the architecture: DeepGAPCNN_v2

7.1 Motivation

Section 5 produced a coherent diagnosis. Every attempt to regularise the baseline made it worse, the train/validation gap was small (2.37 points), and training accuracy under strong augmentation stalled at 90.29%. All three observations point the same way: the model is capacity-limited. If that is correct, the fix is not less augmentation but more parameters — and the augmentations that hurt at 4.7M parameters should start helping once the network can afford them. This section tests that prediction directly.

7.2 Architecture v2

DeepGAPCNN_v2 (Table 9) keeps the block design unchanged and modifies only the capacity schedule: the stem starts at 64 channels instead of 32, an extra stage is inserted, and the progression becomes $64 \rightarrow 128 \rightarrow 256 \rightarrow 384 \rightarrow 512 \rightarrow 512$. The result is 12,215,114 parameters

— $2.6\times$ the baseline — across **13 Conv2d/Linear layers**, still comfortably under the 30-layer ceiling.

The training recipe is also strengthened, since a larger model at batch size 1024 is harder to start. AdamW replaces Adam so that weight decay is decoupled from the adaptive step, and the schedule becomes an 8-epoch linear warm-up followed by cosine decay:

$$\eta_t = \begin{cases} \eta_{\max} \cdot \frac{t+1}{T_w}, & t < T_w, \\ \frac{\eta_{\max}}{2} \left(1 + \cos \left(\pi \frac{t-T_w}{T-T_w} \right) \right), & t \geq T_w, \end{cases} \quad (5)$$

with $T_w = 8$, $T = 120$ and $\eta_{\max} = 2 \times 10^{-3}$. The warm-up addresses exactly the early-epoch instability observed in Section 4.1.

7.3 Baseline v2 versus v2 + MixUp

v2 with mild augmentation. Trained with label smoothing and the baseline augmentation pipeline, v2 reaches a best validation accuracy of 95.95% and a test accuracy of **94.97%**, gaining 2.76 points over v1. Early stopping triggers at epoch 64 after 20 epochs without improvement, in 1,414 s. The training accuracy, however, reaches 99.69%: the model now fits the training set almost perfectly, and the train/validation gap has widened from 2.37 to roughly 4.8 points. The regime has switched from bias-limited to variance-limited.

v2 with strong augmentation and MixUp. The same architecture trained with strong augmentation, label smoothing and MixUp ($\alpha = 0.2$) for the full 120 epochs reaches best validation 97.26% and test **96.92%** — a further +1.95 points, in 2,762 s.

Interpretation. This is the decisive result of the project, and it confirms the prediction of Section 7.1. The identical MixUp + strong augmentation combination that cost 1.6 points on v1 *gains* 1.95 points on v2. Regularisation was never the wrong tool; it was applied to a model that had no variance to remove. Training accuracy under MixUp on v2 fluctuates around 86–94% while validation climbs steadily past 97%, the characteristic signature of a healthily regularised model.

Table 10: Five DeepGAPCNN_v2 models trained with identical hyperparameters and different random seeds, and their ensemble.

Model	Test accuracy
Seed 42 (original)	96.92%
Seed 123	96.76%
Seed 2024	96.81%
Seed 7	96.95%
Seed 999	96.76%
Mean $\pm$ std	96.84 $\pm$ 0.08%
Ensemble (no TTA)	97.39%
Ensemble (+ TTA)	97.37%

7.4 Multi-seed deep ensemble

The final step replaces heterogeneous ensembling (different training recipes) with a deep ensemble [12]: five copies of the best v2 configuration, identical in every respect except the random seed (42, 123, 2024, 7, 999), which controls both initialisation and the augmentation and MixUp draws.

Table 10 reports the outcome. Individual accuracies are remarkably stable — $96.84 \pm 0.08\%$ across five seeds, a spread of only 0.19 points between best and worst — which is itself a useful result: the +1.95-point gain from MixUp in the previous section is an order of magnitude larger than seed noise and is therefore genuine, not a lucky run.

The ensemble reaches **97.39%**, +0.44 points over the best individual seed. The mean pairwise error correlation between seeds is $\rho = 0.610$, higher than the 0.496–0.558 obtained in Section 6.1 from models trained with *different* recipes. This is expected: seeds diversify only the optimisation trajectory, not the objective. The gain per model is consequently smaller than the recipe-diversity ensemble delivered, but it comes from a far stronger base and so ends up higher in absolute terms.

TTA brings nothing at this point (97.37% versus 97.39%, within noise). Consistently with Section 6.2, five models trained with strong augmentation plus MixUp have already absorbed the flip and shift invariances that TTA exploits, and averaging over five members has already removed most of the variance TTA could target.

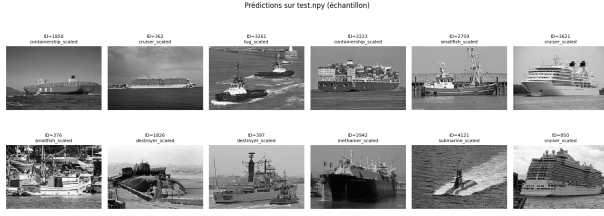


Figure 8: Twelve competition images from `test_images.npy` with the predicted class. Predictions shown are those of the earlier E4 ensemble (Section 6.3).

7.5 Final submission

The final `test.csv` is produced by the five-model v2 ensemble with TTA applied to the 4,224 competition images. The `test_images.npy` array is loaded as `uint8`, scaled to `[0, 1]`, given an explicit channel axis to reach shape `(4224, 1, 128, 192)`, and normalised with the *training* statistics $(\mu, \sigma) = (0.5420, 0.2420)$ so that the competition images undergo exactly the same preprocessing as the validation and test splits.

Figure 8 shows a sample of competition images with their predicted labels; visual inspection reveals no obvious systematic failure. It should be stressed that **no leaderboard score is available** in the experimental record: every figure reported in this document is measured on the internal test split of 3,798 labelled images, which was never used for training or model selection but is drawn from the same distribution as the training data. The competition score may therefore differ.

8 Discussion

8.1 What worked, and what did not

Table 11 collects every configuration. The trajectory from 92.21% to 97.39% is a gain of **+5.18** accuracy points, and its decomposition is more instructive than the total.

Capacity dominated everything. Densifying the architecture contributed +2.76 points on its own, and unlocked a further +1.95 from regularisation that had previously been harmful — 4.71 of the 5.18 total points. By comparison,

Table 11: Complete summary of all configurations evaluated. Accuracies on the internal test split.

Configuration	Accuracy
DeepGAPCNN (v1 baseline)	92.21%
+ label smoothing	92.92%
+ strong augmentation	90.60%
+ MixUp	90.57%
+ CutMix	88.02%
Best single model + TTA	92.79%
Ensemble E1 (4 models)	92.58%
Ensemble E2/E3 (5 models, TTA)	93.26%
Ensemble E4 top-4	93.42%
DeepGAPCNN_v2 baseline	94.97%
DeepGAPCNN_v2 + MixUp	96.92%
v2 multi-seed ensemble (no TTA)	97.39%
v2 multi-seed ensemble (+ TTA)	97.37%

the entire single-model regularisation study of Section 5 netted +0.71 points, and ensembling five heterogeneous v1 models netted +1.21 over the v1 baseline.

Regularisation is conditional, not absolute. The clearest lesson is that MixUp and strong augmentation are neither good nor bad techniques in the abstract: the same configuration cost 1.6 points on a 4.7M-parameter model and gained 1.95 on a 12.2M-parameter one. The relevant question is never “does this regulariser work” but “is this model in a variance-limited regime”. The train/validation gap, not the leaderboard, is the instrument that answers it.

TTA and ensembling are variance-reduction tools. Both delivered their largest gains on the weakest models and near-zero gains on the strongest (TTA: +0.50 on CutMix, −0.71 on the baseline; ensembling: +0.50 on v1, +0.44 on v2 despite five members instead of four). Neither can substitute for a better model.

Diversity is necessary but not sufficient. CutMix was the most complementary ensemble member by error correlation ($\rho = 0.496$) and was still worth excluding, because at 88.02% its confident errors dominated the uniform average.

8.2 Limitations

Several limitations bound the conclusions above.

No external validation. All reported figures come from a single internal split with a single seed for the split itself. No cross-validation was performed, and no leaderboard feedback is recorded. With 3,798 test images, the standard error on an accuracy near 97% is roughly 0.28 points, so differences below about half a point — such as the 0.02 between the v2 ensemble with and without TTA — should be read as noise.

Incomplete ablation. The single-model study is cumulative rather than factorial: strong augmentation was always applied on top of label smoothing, and MixUp on top of both. The individual contribution of strong augmentation alone, or of MixUp without label smoothing, is therefore not measurable from these runs. Likewise, only the v2 + MixUp configuration was carried to the multi-seed stage, so it is unknown whether v2 + CutMix would have become profitable at the higher capacity as MixUp did.

Unexplored directions. Weighted ensemble averaging was never tried, although Section 6.3 suggests it could have recovered CutMix. The class weighting was fixed at inverse frequency and never tuned, even though Section 4.3 shows it is directly responsible for the precision asymmetry on *corvette* and *submarine*. Input resolution was kept at the native 128×192 , but since the residual errors are concentrated in fine superstructure detail, upsampling is a plausible avenue. No architectural element beyond depth and width was varied: residual connections, squeeze-and-excitation and dilated convolutions all fit within the 30-layer budget and were not tested.

Irreducible confusions. The *corvette/destroyer/coastguard* cluster is likely to remain the accuracy ceiling. These are grey-hulled military vessels of overlapping length whose distinguishing features occupy a few pixels at this resolution, and the errors are symmetric — a signature of genuine ambiguity rather than of a biased classifier.

9 Conclusion

This project illustrates how a systematic, one-change-at-a-time methodology converges on the right diagnosis even when the initial hypothesis is wrong.

A hand-written 11-layer convolutional network with global average pooling, trained from scratch under class-weighted cross-entropy, establishes a solid 92.21% baseline on a 10-class greyscale ship classification problem. A conventional regularisation study then fails almost completely: label smoothing adds 0.71 points, while strong augmentation, MixUp and CutMix each cost between 1.6 and 4.2 points. Ensembling and test-time augmentation recover part of the loss, reaching 93.42%, but the pattern of results — gains concentrated on the weakest models, a small train/validation gap, training accuracy stalling under augmentation — points to a bias-limited rather than variance-limited regime.

Acting on that diagnosis by densifying the network to 13 layers and 12.2M parameters yields 94.97%, and re-applying the previously harmful augmentation and MixUp now *gains* 1.95 points for a single model at 96.92%. A five-seed deep ensemble closes at **97.39%**, with a seed-to-seed standard deviation of only 0.08 points confirming the stability of the result. Both architectures remain far inside the 30-layer constraint set by the assignment.

The overall progression from 92.21% to 97.39% represents a gain of +5.18 accuracy points, of which 4.71 are attributable directly or indirectly to the capacity increase. The residual errors are concentrated in a single coherent group of visually similar naval surface combatants, and reflect the limits of a 128×192 greyscale input rather than a deficiency of the optimisation.

References

- [1] M. Lin, Q. Chen, S. Yan, “Network In Network,” *International Conference on Learning Representations (ICLR)*, 2014.
- [2] S. Ioffe, C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” *International Conference on Machine Learning (ICML)*, pp. 448–456, 2015.

- [3] K. Simonyan, A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” *International Conference on Learning Representations (ICLR)*, 2015.
- [4] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.
- [5] D. P. Kingma, J. Ba, “Adam: A Method for Stochastic Optimization,” *International Conference on Learning Representations (ICLR)*, 2015.
- [6] I. Loshchilov, F. Hutter, “Decoupled Weight Decay Regularization,” *International Conference on Learning Representations (ICLR)*, 2019.
- [7] I. Loshchilov, F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” *International Conference on Learning Representations (ICLR)*, 2017.
- [8] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, Z. Wojna, “Rethinking the Inception Architecture for Computer Vision,” *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2818–2826, 2016.
- [9] H. Zhang, M. Cissé, Y. N. Dauphin, D. Lopez-Paz, “mixup: Beyond Empirical Risk Minimization,” *International Conference on Learning Representations (ICLR)*, 2018.
- [10] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, Y. Yoo, “CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features,” *IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 6023–6032, 2019.
- [11] Z. Zhong, L. Zheng, G. Kang, S. Li, Y. Yang, “Random Erasing Data Augmentation,” *AAAI Conference on Artificial Intelligence*, pp. 13001–13008, 2020.
- [12] B. Lakshminarayanan, A. Pritzel, C. Blundell, “Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles,” *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 6402–6413, 2017.
- [13] P. Goyal, P. Dollár, R. Girshick, P. Noordhuis, L. Wesolowski, A. Kyrola, A. Tulloch, Y. Jia, K. He, “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” *arXiv:1706.02677*, 2017.